```
! pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.25.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.8.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (1.4.2)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.28.1)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.24.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2026.2.25)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=202
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=20
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=202
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=20
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datasets) (4.13.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datasets) (1.0.9
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datas
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub>=0.24.0->datas
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub>=0.24.0->dat
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-hub>=0
```

```
from datasets import load_dataset
dataset =  load_dataset("bentrevett/multi30k")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:       1.15k/? [00:00<00:00, 103kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
train.jsonl:       4.60M/? [00:00<00:00, 61.3MB/s]
val.jsonl:       164k/? [00:00<00:00, 10.4MB/s]
test.jsonl:       156k/? [00:00<00:00, 12.5MB/s]
Generating train split: 100%                                    29000/29000 [00:00<00:00, 229569.39 examples/s]
Generating validation split: 100%                                    1014/1014 [00:00<00:00, 50706.09 examples/s]
Generating test split: 100%                                    1000/1000 [00:00<00:00, 47254.44 examples/s]
```

```
train,val,test = dataset["train"],dataset["validation"],dataset["test"]
```

Building the tokenization

```
import torch
import torch.nn as nn
import torch.optim as optim
import random
import numpy as np
import spacy
import datasets
import tqdm
```

```
seed = 1234

random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
```

```
torch.cuda.manual_seed(seed)
torch.backends.cudnn.deterministic = True
```

```
train[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.'}
```

```
!python -m spacy download en_core_web_sm
!python -m spacy download de_core_news_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 112.6 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
Collecting de-core-news-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/de_core_news_sm-3.8.0/de_core_news_sm-3.8.0-py3-none-any.whl (14.6 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.6/14.6 MB 71.9 MB/s eta 0:00:00
Installing collected packages: de-core-news-sm
Successfully installed de-core-news-sm-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('de_core_news_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
## loading the tokenizer
en_nlp = spacy.load("en_core_web_sm")
de_nlp = spacy.load("de_core_news_sm")
```

```
sentence = "hi how are you"
[text.text for text in en_nlp.tokenizer(sentence)]
```

```
['hi', 'how', 'are', 'you']
```

```
def tokenize_examples(example,en_nlp,de_nlp,lower,sos_token,eos_token,max_length):
  english = [token.text for token in en_nlp.tokenizer(example["en"])][:max_length]
  german = [token.text  for token in de_nlp.tokenizer(example["de"])][:max_length]
  if lower:
    english = [text.lower() for text in english]
```

```
    german = [text.lower() for text in german]
  english = [sos_token] + english + [eos_token]
  german = [sos_token] + german + [eos_token]
  return {"en_tokens":english,"de_tokens":german}
```

```
tokenize_examples(train[0],en_nlp,de_nlp,1,"<sos>","<eos>",100)
```

```
{'en_tokens': ['<sos>',
  'two',
  'young',
  ',',
  'white',
  'males',
  'are',
  'outside',
  'near',
  'many',
  'bushes',
  '.',
  '<eos>'],
 'de_tokens': ['<sos>',
  'zwei',
  'junge',
  'weiße',
  'männer',
  'sind',
  'im',
  'freien',
  'in',
  'der',
  'nähe',
  'vieler',
  'büsche',
  '.',
  '<eos>']}
```

```
max_length = 1_000
lower = True
sos_token = "<sos>"
eos_token = "<eos>"

fn_kwargs = {
    "en_nlp": en_nlp,
    "de_nlp": de_nlp,
    "lower": lower,
    "sos_token": sos_token,
    "eos_token": eos_token,
```

```
    "max_length": max_length
}
```

```
train_data = train.map(tokenize_examples, fn_kwargs=fn_kwargs)
valid_data = val.map(tokenize_examples, fn_kwargs=fn_kwargs)
test_data = test.map(tokenize_examples, fn_kwargs=fn_kwargs)
```

```
Map: 100%                                          29000/29000 [00:07<00:00, 4278.54 examples/s]

Map: 100%                                          1014/1014 [00:00<00:00, 4873.74 examples/s]

Map: 100%                                          1000/1000 [00:00<00:00, 4563.77 examples/s]
```

Vocabulary

```
## so lets build the vocabulary
unk_token = "<unk>" # to capture the unknown token while inference
pad_token = "<pad>" # to make the len same
special_tokens = [
    unk_token,
    pad_token,
    sos_token,
    eos_token,
]
```

```
train_data["en_tokens"][0]
```

```
['<sos>',
 'two',
 'young',
 ',',
 'white',
 'males',
 'are',
 'outside',
 'near',
 'many',
 'bushes',
 '.',
 '<eos>']
```

```
from collections import Counter
vocab = {
```

```
    "<unk>":0,
    "<pad>":1,
    "<sos>":2,
    "<eos>":3
}
```

```
Counter
```

collections.Counter
def __init__(self, iterable=None, /, **kwds)

Dict subclass for counting hashable items. Sometimes called a bag
or multiset. Elements are stored as dictionary keys and their counts
are stored as dictionary values.

```
>>> c = Counter('abcdeabcdabcaba')  # count elements from a string
```

```
for sentence in train_data["en_tokens"]:
  for token in sentence:
    if token not in vocab:
      vocab[token] = len(vocab) + 1
  break
```

```
sentence
```

```
['<sos>',
 'two',
 'young',
 ',',
 'white',
 'males',
 'are',
 'outside',
 'near',
 'many',
 'bushes',
 '.',
 '<eos>']
```

```
vocab
```

```
{'<unk>': 0,
 '<pad>': 1,
 '<sos>': 2,
 '<eos>': 3,
```

```
 'two': 5,
 'young': 6,
 ',': 7,
 'white': 8,
 'males': 9,
 'are': 10,
 'outside': 11,
 'near': 12,
 'many': 13,
 'bushes': 14,
 '.': 15}
```

```
en_vocab =  {
    "<unk>":0,
    "<pad>":1,
    "<sos>":2,
    "<eos>":3
}
# english
en_counter = Counter()
for sentence in train_data["en_tokens"]:
    en_counter.update(sentence)
min_freq = 2
for token, count in en_counter.items():
    if count >= min_freq and token not in en_vocab:
        en_vocab[token] = len(en_vocab)
```

```
en_vocab
```

Show hidden output

```
de_vocab =  {
    "<unk>":0,
    "<pad>":1,
    "<sos>":2,
    "<eos>":3
}
# german
de_counter = Counter()
for sentence in train_data["de_tokens"]:
    de_counter.update(sentence)
min_freq = 2
for token, count in de_counter.items():
    if count >= min_freq and token not in de_vocab:
        de_vocab[token] = len(de_vocab)
```

```
en_vocab["the"]
```

```
41
```

```
len(en_vocab),len(de_vocab)
```

```
(5893, 7853)
```

```
# just and test
"The" in en_vocab
```

```
False
```

```
assert en_vocab[unk_token] == de_vocab[unk_token]
assert en_vocab[pad_token] == de_vocab[pad_token]

unk_index = en_vocab[unk_token]
pad_index = en_vocab[pad_token]
# to check that they got the came index
```

```
unk_index , pad_index
```

```
(0, 1)
```

```
en_vocab["The"] # as this the manual vocab building so some methods to use
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_530/2979932904.py in <cell line: 0>()
----> 1 en_vocab["The"]

KeyError: 'The'
```

```
def lookup_indices(example,en_vocab,de_vocab):
  en_indices,de_indices = [],[]
  #english
  for token in example["en_tokens"]:
    if token not in en_vocab:
      en_indices.append(en_vocab["<unk>"])
    else:  # Add this else!
        en_indices.append(en_vocab[token])
  #german
  for token in example["de_tokens"]:
```

```
    if token not in de_vocab:
      de_indices.append(de_vocab["<unk>"])
    else:  # Add this else!
        de_indices.append(de_vocab[token])
  return {"en_ids":torch.tensor(en_indices),"de_ids":torch.tensor(de_indices)}
```

```
lookup_indices(train_data[0],en_vocab,de_vocab)
```

```
{'en_ids': tensor([ 2,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14,  3]),
 'de_ids': tensor([ 2,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  3])}
```

```
train[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.'}
```

```
train_data[0]
```

Show hidden output

```
fn_kwargs = {
    "en_vocab":en_vocab,
    "de_vocab":de_vocab
}
train_data = train_data.map(lookup_indices,fn_kwargs=fn_kwargs)
```

Map: 100% 29000/29000 [00:05<00:00, 5849.78 examples/s]

```
train_data[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.',
 'en_tokens': ['<sos>',
  'two',
  'young',
  ',',
  'white',
  'males',
  'are',
  'outside',
  'near',
  'many',
  'bushes',
  '.',
  '<eos>'],
```

```
 'de_tokens': ['<sos>',
  'zwei',
  'junge',
  'weiße',
  'männer',
  'sind',
  'im',
  'freien',
  'in',
  'der',
  'nähe',
  'vieler',
  'büsche',
  '.',
  '<eos>'],
 'en_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 3],
 'de_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 3]}
```

```
valid_data = valid_data.map(lookup_indices,fn_kwargs=fn_kwargs)
test_data = test_data.map(lookup_indices,fn_kwargs=fn_kwargs)
```

```
Map: 100%                                          1014/1014 [00:00<00:00, 3314.57 examples/s]

Map: 100%                                          1000/1000 [00:00<00:00, 4206.61 examples/s]
```

```
# reverse vocab
en_itos = {i: s for s, i in en_vocab.items()}
de_itos = {i: s for s, i in de_vocab.items()}

def lookup_vocab(example,en_itos,de_itos):
  en_indices,de_indices = [],[]
  #english
  for ids in example["en_ids"]:
    en_indices.append(en_itos[ids])
  #german
  for ids in example["de_ids"]:
    de_indices.append(de_itos[ids])
  return {"en_tokens_rev":(en_indices),"de_tokens_rev":(de_indices)}
```

```
train_data
```

```
Dataset({
    features: ['en', 'de', 'en_tokens', 'de_tokens'],
    num_rows: 29000
})
```

```
lookup_indices(train_data[0],en_vocab,de_vocab)
```

```
{'en_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 3],
 'de_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 3]}
```

```
lookup_vocab(lookup_indices(train_data[0],en_vocab,de_vocab),en_itos,de_itos)
```

```
{'en_tokens_rev': ['<sos>',
  'two',
  'young',
  ',',
  'white',
  'males',
  'are',
  'outside',
  'near',
  'many',
  'bushes',
  '.',
  '<eos>'],
 'de_tokens_rev': ['<sos>',
  'zwei',
  'junge',
  'weiße',
  'männer',
  'sind',
  'im',
  'freien',
  'in',
  'der',
  'nähe',
  'vieler',
  'büsche',
  '.',
  '<eos>']}
```

```
train_data[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.',
 'en_tokens': ['<sos>',
  'two',
  'young',
  ',',
  'white',
  'males',
  'are',
  'outside',
  'near',
```

```
     'many',
     'bushes',
     '.',
     '<eos>'],
    'de_tokens': ['<sos>',
     'zwei',
     'junge',
     'weiße',
     'männer',
     'sind',
     'im',
     'freien',
     'in',
     'der',
     'nähe',
     'vieler',
     'büsche',
     '.',
     '<eos>']}
```

Data Loader

```
def get_collate_fn(pad_index):
    def collate_fn(batch):
        batch_en_ids = [torch.tensor(example["en_ids"]) for example in batch]
        batch_de_ids = [torch.tensor(example["de_ids"]) for example in batch]
        batch_en_ids = nn.utils.rnn.pad_sequence(batch_en_ids, padding_value=pad_index) # padding as the same length for one batch
        batch_de_ids = nn.utils.rnn.pad_sequence(batch_de_ids, padding_value=pad_index)
        batch = {
            "en_ids": batch_en_ids,
            "de_ids": batch_de_ids,
        }
        return batch

    return collate_fn
```

```
def get_data_loader(dataset, batch_size, pad_index, shuffle=False):
    collate_fn = get_collate_fn(pad_index)
    data_loader = torch.utils.data.DataLoader(
        dataset=dataset,
        batch_size=batch_size,
        collate_fn=collate_fn,
        shuffle=shuffle,
    )
    return data_loader
```

```
batch_size = 32

train_data_loader = get_data_loader(train_data, batch_size, pad_index, shuffle=True)
valid_data_loader = get_data_loader(valid_data, batch_size, pad_index)
test_data_loader = get_data_loader(test_data, batch_size, pad_index)
```

```
for x in train_data_loader:
  break
```

```
len(x["en_ids"]) , len(x["de_ids"])
```

```
(22, 22)
```

Next model training